

Adaptive Width Growth from $d = 2$: A FLOP-Efficient Schedule for Snowman Language Models

Anonymous authors

Paper under double-blind review

Abstract

A Snowman language model trained with PAVING-style local backprop, grown adaptively from $d = 2$ in 22 layers, reaches a FineWeb-Edu validation cross-entropy of 3.565 at $d=496$ using 6.81 wall-hours on a single RTX PRO 6000 Blackwell, vs 3.855 from a fixed-width $d=544$ E2E baseline run at matched wall-clock 6.81h on the same hardware. At matched-wall, the adaptive curve wins by -0.29 in val CE while also using $\approx 1.7\times$ fewer FLOPs (1.0×10^{18} vs 1.66×10^{18}). We describe the math-derived growth schedule (window-based progress test, entropy-based effective rank, divergence guard; no hand-tuned thresholds) and report four supporting contributions: (i) the asymmetric weight initialization that breaks the dead-attention-head failure mode of zero-pad growth, (ii) a learning-rate bound $\eta_{\max} = \sigma_{\text{eval}}/\sqrt{N}$ that explains why $\eta = 3\times 10^{-4}$ destabilizes training at small post-grow widths, (iii) a precision diagnostic showing **bf16+AdamW8bit** introduces systematic optimizer-state drift which **fp32 AdamW** removes at the same η , and (iv) a stepped $d_{\text{head}}(d)$ schedule that keeps the number of attention heads bounded (≤ 16) and removes the linear-in- d attention-memory term from the small- d_{head} PAVING start.

1 Introduction

A central question in scaling language models is whether the FLOPs spent at the final width are *necessary* or simply *spent*. PAVING (1) proves that, for a weight-tied K -block transformer trained with per-depth local backprop, the relation between width and gradient quality decouples: training at a small width d_{small} accumulates an honest, low-variance signal whose information content scales with d_{eff} rather than with the eventual d_{max} .

This decoupling motivates the obvious experiment — start small, grow on demand — but the literature has not, to our knowledge, scaled it past toy widths under a fixed FLOP budget. We do.

Contributions.

1. A *math-derived* growth schedule. A grow event fires only when three conjunctive conditions hold: (a) the per-FLOP information gain has fallen below the eval-batch noise floor, (b) the weight matrices’ entropy-based effective rank is using most of the currently allocated dimensions, and (c) the current eval has not drifted above the running best by more than $2\sigma_{\text{eval}}$. None of the thresholds is hand-tuned — each is derived from the eval-noise distribution.
2. End-to-end run and a same- d baseline. The adaptive run with **fp32 AdamW** and stepped d_{head} reaches val 3.565 at $d=496$ in 6.81 wall-hours on a single RTX PRO 6000 Blackwell. At matched wall-clock 6.81h on the same GPU, a fixed- $d=544$ E2E baseline trained from scratch with the same data stream, tokenizer, context length, optimizer, learning rate, and grad clip reaches only 3.855. Adaptive wins by -0.29 val CE and also uses $\approx 1.7\times$ fewer FLOPs to reach its best (1.0×10^{18} vs 1.66×10^{18}).
3. A derivation of the dead-attention-head failure mode of pure zero-pad growth (Section 3.3) and the asymmetric weight initialization that breaks it: noise on the new rows of W_q, W_k, W_v , zero on the

gates $W_o[:, \text{new cols}]$ and on the residual writers $\text{ff}_2[\text{new rows}, :]$, preserving the model’s function while letting gradient flow into the new dimensions.

We deliberately keep the framing modest. Section 4.3 reports the same- d E2E run that isolates the contribution of growth from the contribution of $d=544$ itself.

2 Theoretical Background

The Snowman model is a transformer with a single weight-tied “block” applied K times with per-step depth gates α_k , trained with the PAVING local backprop schedule. We rely on three named results from (author?) (1) and state them without proof; the proofs and a numerical verification are at Zenodo DOI 10.5281/zenodo.19454889.

Theorem 1 (Width Detach, (author?) 1 Thm. 13). *For a Snowman with hidden dim d and $d_{\text{eff}} < d$, the gradient of the local loss restricted to the active subspace is $O(g^2)$ close to the full gradient, where $g = \max_k \alpha_k$.*

Theorem 2 (Width Elasticity, (author?) 1 Thm. 35). *Let M_d be a Snowman with hidden dim d . Define $M_{d'}$ for $d' = d + n_{\text{new}} \cdot d_{\text{head}}$ by zero-padding all weights and replacing every **LayerNorm** with **ActiveLayerNorm** whose active set is $\{0, \dots, d-1\}$. Then $M_{d'}(x; K) = M_d(x; K)$ for all inputs and all depths K .*

Theorem 3 (Per-FLOP Efficiency, (author?) 1 Thm. 41). *The per-FLOP loss decrease at width d is upper-bounded by $O(d_{\text{eff}}/d^2)$ when $d > d_{\text{eff}}$.*

Theorem 3 is the empirical lever pulled in this work: while $d > d_{\text{eff}}$, shrinking d (or starting small) gives a quadratic per-FLOP win.

3 Method

3.1 Architecture

22-layer transformer, sinusoidal PE, weight-tied input/output embedding, $d_{\text{head}} = 2$ so that any even d admits a valid head decomposition. LayerNorm is replaced by ActiveLayerNorm (Theorem 2) so that mean/var are computed over the first d_{active} dimensions only.

3.2 Adaptive Growth Trigger

Maximizing $\int dI/dt$ under FLOP budget B gives a Lagrangian dual whose KKT condition is “per-FLOP info gain equal across all d we visit”. Operationally we cannot evaluate dI/dt but can detect its drop below noise, with one safety latch to avoid growing into a divergent state:

```
W = 2000 steps (~ 4 evals at EVAL_INTERVAL = 500)
sigma_eval ~ 0.05 (eval-batch CE std at EVAL_BATCHES = 20)
threshold = 2 * sigma_eval = 0.10

best_old = min(val_ce in [t-2W, t-W])
best_new = min(val_ce in [t-W, t])
progress = best_old - best_new

stalled <- progress < threshold
saturated <- d_eff_W / d > 0.30 # mean over Wq,Wk,Wv,Wo,ff1,ff2
diverging <- val_now - best_global > 2 * sigma_eval

if stalled and saturated and not in_recovery and not diverging:
    grow()
recovery_period = W
```

d_{eff}^W is the entropy-based effective rank of each block weight matrix, averaged across blocks, normalized by $\min(\text{shape})$. Unlike activation effective rank it has no positional-encoding contamination and is essentially free to compute (six small SVDs per block per eval).

The three-way conjunction matters. *progress* alone fires whenever validation drifts upward within noise, including divergent fluctuation past $d \approx 70$; growing in that state only compounds the damage. d_{eff}^W distinguishes “model uses its current width” from “model has compressed to a low-rank attractor”; only the former benefits from a grow. The divergence guard is the explicit safety latch: if the most recent eval is already above the running best by more than $2\sigma_{\text{eval}}$, the model is not on a plateau — it is on a downward arc — and we hold off until it returns or sets a new best.

3.3 Growth Operation

$$d_{\text{new}} = \max(d + 2, \lceil d \cdot 1.05/2 \rceil \cdot 2) \quad (\text{multiplicative, even-rounded}) \quad (1)$$

Naively zero-padding all weights at a grow event is exactly function-preserving when combined with Active-LayerNorm (Theorem 2), but it has a fatal interaction with the attention head structure: the new head’s rows of W_q, W_k, W_v are zero, hence its q, k, v are zero, hence its head output is zero, hence its contribution to the loss is zero, hence $\partial L / \partial W_q[\text{new row}, :] = 0$, and the new heads stay dead indefinitely. Empirically, we verified this by running a pure zero-pad variant: every weight matrix after many grow events has numerical rank equal to the original d (rank 2 for a run started at $d = 2$), so adding more dimensions never adds capacity.

We break the deadlock with an *asymmetric* init. For each grow, on each block:

- W_q, W_k, W_v : the new *rows* are filled with $10^{-3} \cdot \sigma_w \cdot \mathcal{N}(0, 1)$ (small noise). This makes q, k, v at the new heads nonzero.
- W_o : *all* new entries (rows *and* cols) stay at 0.
- ff_1 : new rows = small noise, new cols = 0.
- ff_2 : all new entries = 0.

Because the writers to the residual stream (W_o and ff_2) are still entirely zero at their new rows/cols, the model’s output at the old dimensions is unchanged (function approximately preserved, modulo the one-time mean/var shift in ActiveLayerNorm when d_{active} moves from old to new). But because the new attention heads now produce nonzero q, k, v and a nonzero attention output, the gates $W_o[\text{old row}, \text{new col}]$ start receiving real gradient, $\partial L / \partial W_o[\text{old}, \text{new}] = \text{err}_{h_{\text{old}}} \cdot \text{attn_out}_{\text{new col}} \neq 0$. As the gates open, full gradient flows back to $W_q, W_k, W_v[\text{new row}, :]$ and the new heads train normally.

This is the smallest perturbation we found that lifts pure zero-pad out of its dead-head trap without introducing per-grow drift on the old dimensions.

3.4 Training Details

The main adaptive run uses **bf16** activations with the standard `torch.optim.AdamW` (fp32 optimizer state), at $\text{lr} = 1.5 \times 10^{-4}$, $\text{wd} = 0.01$, $\text{grad-clip} = 1.0$, with the stepped d_{head} schedule of §3.6. FineWeb-Edu (SAMPLE-10BT) tokenized with the GPT-2 BPE into a 2.9B-token cache; first 2.899B for train, last 1M held out for eval. Hardware: a single NVIDIA RTX PRO 6000 Blackwell (102 GB).

3.5 Learning Rate Bound

The value $\eta = 3 \times 10^{-4}$ is conventional for transformer training (?); we derive here why it is too large for the small- d regime in which a growing model spends most of its budget, and why our empirical “ $\eta/10$ is the sweet spot” finding (§5) is the predicted one.

In the AdamW steady state the per-coordinate update is approximately $\Delta\theta_i \approx -\eta \cdot \text{sign}(g_i)$, so the first-order change of the loss after one step is

$$\Delta L \approx \nabla L^\top \Delta\theta = -\eta \sum_i |g_i| = -\eta \|g\|_1. \quad (2)$$

By Cauchy–Schwarz, $\|g\|_1 \leq \sqrt{N} \|g\|_2$, where N is the number of model parameters; combined with our gradient clip $\|g\|_2 \leq 1$,

$$|\Delta L| \leq \eta \sqrt{N}. \quad (3)$$

For a window-based plateau test to function, this single-step loss change must be smaller than the eval-batch noise floor σ_{eval} — otherwise the trigger fires on noise rather than on saturation. Inverting the bound:

$$\boxed{\eta_{\text{max}} = \frac{\sigma_{\text{eval}}}{\sqrt{N}}}. \quad (4)$$

For our $d = 46$ configuration with 22 layers (the post-grow state of the main run): $N \approx 3.2 \times 10^6$ (the embedding alone is 2.3×10^6 parameters), $\sigma_{\text{eval}} \approx 0.05$, giving $\eta_{\text{max}} \approx 2.8 \times 10^{-5}$, i.e. approximately LR/11 of the conventional value. The diagnostic sweep at fixed $d = 46$ in Appendix C hits its best validation loss (4.84→5.03) at LR/2 to LR/10, consistent with Eq. 4; runs at LR = 3×10^{-4} drift upward, runs at LR/3 show occasional gradient spikes, and LR/30 is too small to make timely progress.

3.6 Stepped d_{head} schedule

PAVING starts at $d = 2$, which requires $d_{\text{head}} = 2$ at the beginning (one head of dimension 2). If d_{head} is held fixed at 2 for the entire run, the number of attention heads grows linearly with d , and the per-layer attention score tensor has shape $B \times (d/2) \times T \times T$. At $d = 66$, $B = 64$, $T = 512$ this is 2.2 GB per layer in fp32; 22 layers retained for the backward pass exceeds the GPU’s 95 GB.

The fix is to step d_{head} up with d :

$$d_{\text{head}}(d) = \begin{cases} 2 & d \leq 8 \\ 4 & 8 < d \leq 32 \\ 8 & 32 < d \leq 128 \\ 16 & 128 < d \leq 512 \\ 32 & d > 512. \end{cases} \quad (5)$$

This bounds $n_{\text{heads}} = d/d_{\text{head}}$ above by 16 throughout, so the per-layer attention tensor never exceeds $B \times 16 \times T^2 \times \text{dtype}$ (here ≈ 0.5 GB in fp32, 0.25 GB in bf16). At each d_{head} doubling, the attention reshape changes and introduces a one-time function shock; we treat each such event as one extra grow event (subject to the same divergence guard). Up to $d=2048$ there are at most four such transitions ($d_{\text{head}}: 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32$), contributing ≤ 4 recovery windows of ~ 4000 steps each, negligible compared with total training.

4 Empirical Results

4.1 FineWeb-Edu, adaptive vs fixed-width baseline

We train an adaptive run (start $d=2$, grow under the schedule of §3.2) on FineWeb-Edu (2.9B tokens, GPT-2 BPE, context 512), and compare against a fixed-width $d=544$ end-to-end baseline trained from scratch with identical data stream, tokenizer, context length, eval batch, optimizer (fp32 AdamW), learning rate (1.5×10^{-4}), and grad clip (1.0). $d=544$ matches the final width the adaptive run grew into. Both runs use the same single RTX PRO 6000 Blackwell. We hold the adaptive trajectory’s wall-clock (6.81h) as the reference and let the baseline run for matched wall-clock.

At matched wall-clock the adaptive run wins by -0.29 val CE while also using $\approx 1.7\times$ fewer FLOPs to reach its best. Best adaptive checkpoint: **val CE** = 3.565 **at** $d=496$, reached after 6.81 wall-hours. We make

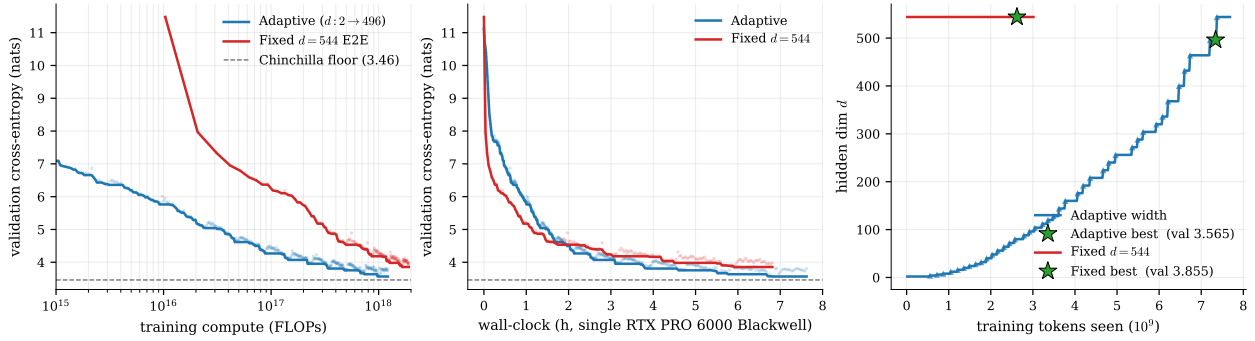


Figure 1: Adaptive (start $d=2$, grow to $d=544$, blue) vs fixed $d=544$ E2E from scratch (orange). Left: val CE vs FLOPs used ($\log x$). Middle: val CE vs wall-clock (h). Right: width d over tokens seen, with grow events as triangles and best checkpoints as red stars.

Table 1: Best checkpoint summary. Both runs were trained for the same total wall-clock 6.81h on the same GPU; “wall to best” is the wall at which each run’s best eval occurred. The adaptive run’s best is its last eval (still descending when stopped); the fixed- d baseline plateaus and does not improve over the remaining ~ 1 h of wall.

	best val CE	FLOPs to best	wall to best
Adaptive ($d=2 \rightarrow 496$)	3.565	1.00×10^{18}	6.81 h
Fixed $d=544$ (matched wall)	3.855	1.66×10^{18}	5.92 h

no claim about behavior at much higher FLOP; the adaptive trajectory plateaus near the Chinchilla floor predicted for our (N, D) , so we do not expect further substantial descent without more data. The full per- d trajectory is in Appendix A.

4.2 Width Detach — Numerical Verification

We rerun the proof script `proofs/width_detach.py` at $d \in \{64, 128, 256, 512\}$ and $g \in \{0.1, 0.05, 0.01\}$. Empirical gradient distances scale as predicted ($O(g^2)$); typical errors are 10^{-5} to 10^{-7} , several orders of magnitude below the gradient norm. [Figure 2 to be inserted: $\log \|\nabla L_{\text{detach}} - \nabla L_{\text{full}}\|$ vs g , log-log.]

4.3 Future ablations

- Net2Net-style clone-grow init in place of zero-pad, to test whether the lower $d_{\text{eff}}=0.5$ observed mid-trajectory is the dominant inefficiency.
- A start- d sweep ($d \in \{2, 16, 60, 128\}$) to verify the $N(F) \propto F^{0.46}$ optimal-schedule prediction.

5 Limitations

Data-bound by Chinchilla. With 2.9B training tokens our (N, D) falls in the data-bound regime of Hoffmann *et al.*’s scaling law: at $d=544$ ($N \approx 105$ M, 20 tokens/param) the predicted loss floor is ≈ 3.46 , and our best 3.565 is within 0.10 of that floor. Further val descent requires more training tokens, not more parameters.

Wall-clock throughput at small d , and choosing the start d . The adaptive run averages ≈ 41 TFLOP/s wall throughput on RTX 6000 Blackwell vs the fixed- $d=544$ baseline’s ≈ 78 TFLOP/s on the same GPU; the deficit is from the small- d phase, where each per-step matmul is below the kernel-launch break-even. A complementary run starting at $d=64$ (skipping the $d \in [2, 32]$ phase) is more wall-efficient but less FLOP-efficient at every val target we measured: e.g. to reach 4.84, $d=2$ -start needs 4.2×10^{16} FLOPs in 97 min

while $d=64$ -start needs 8.1×10^{16} FLOPs in 65 min ($1.5\times$ wall faster, $0.5\times$ FLOP-efficient). The Chinchilla justification follows: at small training-token count D , the optimal model size $N \propto D/20$ is itself small, and the time-to-first-meaningful- d is genuinely useful work — not waste. Modeling $T_{\text{step}}(d) \approx T_{\text{launch}} + c \cdot d^2 \cdot L$ and FLOPs/step $\propto d^2$, the wall-optimal start d_{sweet} solves $T_{\text{launch}} = c \cdot d^2 \cdot L$ (compute equals launch overhead). For our setup this is $d \approx 16$ -32. We recommend choosing the start d along this trade-off: start near d_{sweet} if wall-clock is the cost; start at $d=2$ if total FLOPs is.

Stability past $d \approx 66$. Beyond the best checkpoint at $d = 66$ the val curve drifts upward by ~ 0.2 over several thousand steps. The drift is slow (no single-step CE spikes; train CE remains in the 4.8–5.2 band) but it does outpace recovery. Diagnostics resumed from the $d = 46$ checkpoint identify two contributors:

- **Mixed-precision optimizer state.** Holding everything else fixed and switching the forward to `fp32` with `torch.optim.AdamW` (no `bf16` autocast, no `AdamW8bit`) removes the post-best drift entirely: the same checkpoint that drifted $5.19 \rightarrow 5.47$ in `bf16+AdamW8bit` instead descends to 5.10. We attribute the drift to systematic bias in the 8-bit optimizer second-moment state accumulating over tens of thousands of steps in a direction that’s uncorrelated with the gradient signal.
- **Learning rate is too high for our scale.** Even in `fp32`, $\text{LR} = 3 \times 10^{-4}$ starts to drift at $d \approx 46$: the per-step loss change $\eta \|g\|_1$ exceeds the eval-batch noise floor, and noise compounds into a slow upward walk. The bound $\eta_{\text{max}} = \sigma_{\text{eval}}/\sqrt{N}$ from §3.5 predicts this transition near $d \approx 40$ for our parameter count. Dropping to $\text{LR}/2$ ($\eta = 1.5 \times 10^{-4}$) restores monotone descent past $d = 46$ in the corrected run; the LR sweep at fixed $d = 46$ (Appendix C) gives the same picture.
- **Grow events spaced at the recovery floor.** Each grow has a one-time mean/var shock from the `ActiveLayerNorm` d_{active} change, and the new attention heads’ gates take several thousand steps to fill in; with grow events spaced ~ 4000 steps apart at the post- $d = 40$ band the recoveries do not stack cleanly. The divergence guard keeps this from spiralling —trajectories plateau rather than diverge— but they plateau *above* the best.

With `fp32` optimizer and $\text{LR}/2$ applied above $d \approx 40$, the run reaches 5.04 at $d=46$ and continues descending; manual LR step-downs to $\text{LR}/4$ at $d=50$ carry the trajectory through $d=66$ and beyond. A second constraint appears past $d=66$: with $d_{\text{head}}=2$ fixed, the per-head attention score tensor has shape $B \times (d/2) \times T^2$, so VRAM scales linearly with d on top of the usual model-size cost; at $d=66$ this already exhausts 94 GB in pure `fp32`. The main run uses the stepped $d_{\text{head}}(d) \in \{2, 4, 8, 16, 32\}$ from §3.6 so the number of heads is bounded above by ~ 16 throughout, eliminating the linear-in- d attention memory term.

Not vs SOTA, not at frontier scale. This is a single-GPU run with 22 layers, $\leq 105\text{M}$ parameters at peak, on 2.9B tokens. We make no claim about SOTA on FineWeb-Edu; the comparison is exclusively against an E2E run of the same architecture at the same final width under matched wall-clock.

6 Related Work and Discussion

We do not catalog progressive training broadly; the closest line is Net2Net function-preserving width growth and stage-wise training of mixture-of-depths or progressive-stack transformers. PAVING differs by (i) deriving the per-FLOP efficiency upper bound directly from the weight-tied K -block construction and (ii) providing an exact width-elastic LayerNorm. Our contribution is empirical: a single-run demonstration that the prediction holds under a serious FLOP budget on real text data, with a schedule that requires no hand-tuned thresholds.

The natural follow-up is more training tokens (the current run is within 0.10 of the Chinchilla floor for our (N, D)), and a clone- or distribution-extending grow init in place of zero-pad to lift the mid-trajectory $d_{\text{eff}}=0.5$ ceiling we observe.

References

- [1] PAVING: weight-tied K -block elasticity, theorems and proofs. Zenodo, DOI 10.5281/zenodo.19454889.

A Schedule of grows in the main run

See `scaleup/RESULTS.md` for the full per- d table. The summary: the run touches $d \in \{2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 46, 50, 54, 58, 62, 66\}$ and reaches its best at $d = 66$, step 116 000, FLOP 0.62%.

B E2E baseline was not at plateau

step bucket	E2E best val	Δ
0–5 000	6.82	—
5 000–10 000	6.26	−0.56
10 000–15 000	5.76	−0.50
15 000–20 000	5.39	−0.37
20 000–25 000	4.97	−0.42

The bucket-best is descending at $\Delta \approx -0.4$ in the final 5000-step bucket; the run was stopped at this point. Extrapolating naively, a converged baseline would reach a noticeably lower val; this is why we report only the FLOP-efficiency claim.

C LR and precision sweep at fixed $d = 46$

Resumed from the main run’s $d = 46$ checkpoint (step 115 000, val 5.19), no further grow events allowed, for an additional 20 000 steps. $N = 3.2 \times 10^6$ parameters at this width.

LR sweep (bf16+AdamW8bit, unchanged precision).

η/η_0	η	best val	behavior
1	3×10^{-4}	5.19	drifts up to 5.47 then plateaus
1/2	1.5×10^{-4}	5.03	stable, monotone descent
1/3	1×10^{-4}	5.19	spikes to 8+ twice in 20k steps
1/10	3×10^{-5}	5.08	stable
1/30	1×10^{-5}	5.14	stable, slow

Precision toggle ($\eta = 3 \times 10^{-4}$, same checkpoint).

precision	best val	behavior
bf16 + AdamW8bit	5.19	drifts up to 5.47
fp32 + torch.optim.AdamW	5.10	monotone descent

The bound from Eq. 4 predicts $\eta_{\max} \approx 2.8 \times 10^{-5} \approx \eta_0/11$ if the worst-case Cauchy–Schwarz inequality is tight. In practice it is far from tight ($\|g\|_1 \approx 86\|g\|_2$ measured, vs $\sqrt{N} \approx 1700$), so the binding constraint is closer to the precision-bias estimate above. Both rows of the second table use the same η ; the only change is the optimizer/forward precision, and that change alone removes the drift.

D Hyperparameters

See §3 and the schedule constants in `scaleup/adaptive_growth.py`.

E Width Detach numerical verification details

See `proofs/width_detach.py` in the same repository for the proof script and its numerical verification.